

1. PRÉSENTATION DU PROJET

1.1 Contexte et Problématique

Au Cameroun et dans l'ensemble de l'Afrique subsaharienne, la perte de documents d'identité (CNI, passeport, permis de conduire, cartes bancaires, cartes d'étudiant) est un problème quotidien majeur. Des dizaines de milliers de documents perdus ne retrouvent jamais leur propriétaire légitime, non faute de bonnes volontés, mais faute d'un mécanisme fiable, sûr et moderne pour connecter perdants et retrouveurs.

Les canaux actuels — dépôts dans les commissariats, annonces sur Facebook, affichage physique — souffrent de lacunes criantes : absence totale de vérification d'identité, risques élevés d'usurpation et de fraude, délais excessifs, aucune traçabilité post-remise.

1.2 Vision & Mission

ReTurn (nom commercial) / **DocRetour** (nom technique) est une plateforme mobile de mise en relation intelligente et sécurisée entre propriétaires de documents perdus et personnes ayant trouvé ces documents. La mission est triple :

- **Connecter** rapidement et intelligemment les bonnes personnes grâce à un algorithme de matching multi-critères.
- **Vérifier** l'identité du réclamant avant toute remise pour éliminer la fraude.
- **Orchestrer** une restitution traçable avec preuve photographique et évaluation mutuelle.

1.3 Objectifs Mesurables

Objectif	Indicateur	Cible
Taux de match réussi	Déclarations avec ≥1 match dans les 72h	≥ 40 %
Délai moyen de restitution	Entre création du match et restitution confirmée	< 48 h
Taux de fraude	Restitutions contestées post-vérification	< 1 %
Satisfaction utilisateur	Note moyenne app store	≥ 4.5 / 5
Rétention M1	% utilisateurs actifs 30j post-inscription	≥ 55 %

1.4 Utilisateurs Cibles

Persona	Profil	Besoins Clés
Propriétaire perdant	Toute personne ayant égaré un document officiel	Retrouver rapidement, être averti, prouver qu'il est le propriétaire légitime

Persona	Profil	Besoins Clés
Retrouveur	Personne ayant trouvé un document d'autrui	Déclarer sans friction, être mis en relation, être rassuré sur la légitimité du réclamant
Administrateur	Équipe opérationnelle ReTurn	Modérer, gérer les utilisateurs, suivre les KPIs, configurer les zones et paramètres

2. ARCHITECTURE TECHNIQUE

2.1 Vue d'Ensemble

Couche	Technologie	Rôle
 Mobile	Flutter 3 / Dart	Application iOS & Android
State Mgmt	Riverpod + Riverpod Generator	Gestion réactive et typée de l'état
Navigation	GoRouter 7+	Routage déclaratif avec deep-links
Modèles	Freezed + json_serializable	Immutabilité, sérialisation JSON typée
Auth Mobile	Firebase Auth SDK	OTP SMS + Google Sign-In
Stockage local	flutter_secure_storage + Hive	Tokens sécurisés, préférences offline
HTTP Client	Dio + Retrofit	Appels API REST avec intercepteurs
Biométrie	local_auth	Fingerprint / Face ID — verrouillage app
 API Backend	FastAPI (Python 3.12)	API REST async + WebSocket
ORM	SQLAlchemy 2.x Async	Modèles relationnels + migrations Alembic
Base de données	PostgreSQL 16 (schéma <code>docretour</code>)	Persistance principale
Cache / Queues	Redis 7	Sessions, files de notifications FCM async
Stockage fichiers	MinIO (S3-compatible)	Photos déclarations, selfies vérification
Notifications Push	Firebase Cloud Messaging (FCM)	Push sur match, message, restitution
Auth Backend	Firebase Admin SDK + JWT (JOSE)	Vérification idToken Firebase, JWT sessions
Migrations DB	Alembic	Versioning schéma PostgreSQL
Infrastructure	Docker / Docker Compose	Conteneurisation backend + services
 Admin	HTML + Vanilla JS + Chart.js	Panneau d'administration web

3. MODULES FONCTIONNELS

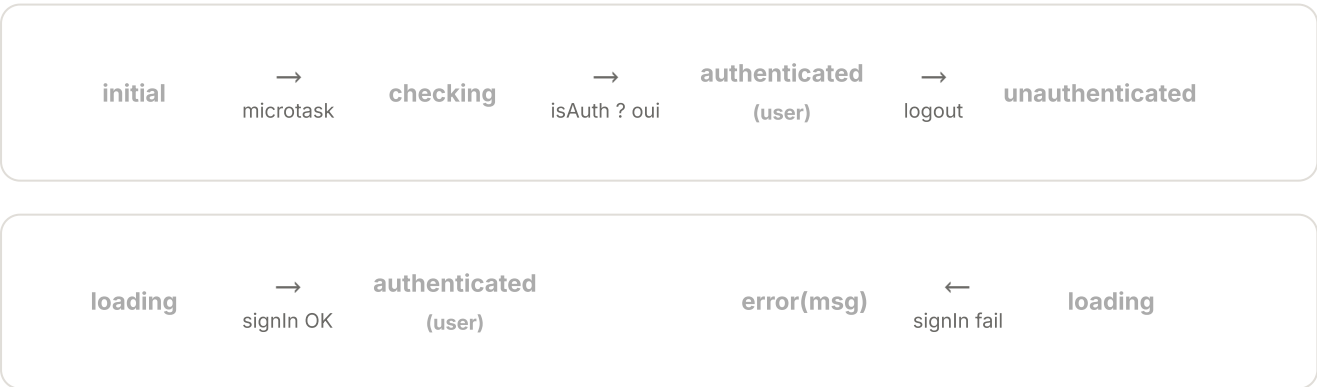
3.1 Module Authentification (AUTH)

Le module AUTH est le point d'entrée de l'application. Il gère l'identification via deux méthodes complémentaires : **OTP SMS** (Firebase Phone Auth) et **Google Sign-In**. Les deux aboutissent au même flux : idToken Firebase → échange contre JWT backend (access + refresh).

Flux OTP SMS — Étape par Étape

#	Action Utilisateur	Traitement Système
1	Saisit son numéro camerounais	Normalisation E.164 (+237XXXXXXXX), appel <code>Firebase.verifyPhoneNumber()</code>
2	Reçoit le SMS et saisit le code OTP (6 chiffres)	Android : récupération auto possible (verificationCompleted)
3	—	<code>signInWithCredential()</code> → idToken Firebase obtenu
4	—	<code>POST /auth/firebase</code> avec idToken → backend vérifie via Firebase Admin SDK, crée ou retrouve l'utilisateur en base, retourne <code>access_token</code> + <code>refresh_token</code>
5	—	Tokens stockés en SecureStorage. <code>AuthNotifier.checkAuth()</code> → <code>GET /auth/me</code> → état <code>authenticated(user)</code>

Diagramme d'État — AuthState



États OTP parallèles : idle → sending → sent(phone) → verifying → verified | error(msg)

⚠ Problèmes Identifiés — AUTH

- Aucune validation côté client du format numéro avant envoi (le backend normalise mais l'UX devrait guider dès la saisie).

- Le `serverClientId` Google est hardcodé dans le code Flutter — à externaliser en variable d'environnement.
- **Pas de refresh_token automatique** : si l'`access_token` expire, l'utilisateur est déconnecté sans tentative de renouvellement silencieux.

✔ Solutions Recommandées

- Ajouter un `InputFormatter` sur le champ téléphone pour forcer `+237` et valider en temps réel.
- Déplacer `serverClientId` dans `.env` / `app_config.dart`.
- Implémenter un intercepteur Dio qui intercepte les 401, appelle `POST /auth/refresh` et rejoue la requête.

3.2 Module Profil Utilisateur (PROFILE)

Le profil est central : il alimente le matching (nom), la vérification d'identité (nom, date de naissance), la confiance (score réputation) et la légitimité du compte.

Modèle de Données Complet — Entité User

Champ	Type	is_profile_complete ?	Formulaire UI actuel ?	Notes
<code>full_name</code>	String(100)	✔ Requis	✔ Oui	Verrouillé une fois renseigné
<code>phone_number</code>	String(100)	✔ Requis (si pas d'email)	✔ Lecture seule si OTP	Format E.164, unique
<code>email</code>	String(254)	✔ Requis (si pas de tel)	✔ Lecture seule si Google	Unique
<code>date_of_birth</code>	Date	✔ Requis	● ABSENT	Manquant dans le formulaire !
<code>gender</code>	String(20)	✔ Requis	● ABSENT	Manquant dans le formulaire !
<code>national_id_number</code>	String(50)	—	● ABSENT	Utile pour la vérification
<code>city</code>	String(100)	✔ Requis	✔ Oui	Améliore le score géo
<code>region / address</code>	String	—	✔ Oui (adresse)	Contexte géographique

Champ	Type	is_profile_complete ?	Formulaire UI actuel ?	Notes
avatar_url	String(512)	—	✅ Tap avatar	URL MinIO
score_reputation	Float (0–10)	—	Lecture seule	Défaut 5.0, mis à jour post-restitution
fcm_token	String(512)	—	Auto	Mis à jour à chaque lancement
is_admin / is_banned / is_active	Boolean	—	Admin seulement	Drapeaux de contrôle

● **Bug Critique — Profil Impossible à Compléter**

- La propriété `is_profile_complete` exige : `full_name` + `(phone|email)` + `date_of_birth` + `gender` + `city` .
- Or le formulaire actuel (`_EditProfileSheet`) **ne propose PAS** `date_of_birth` ni `gender` .
- Résultat : le badge affiche « Incomplet » en permanence. L'utilisateur ne peut **jamais** atteindre « Complet ».
- Impact : toute fonctionnalité conditionnée à un profil complet reste inaccessible.

✅ **Solution — Formulaire Profil en 3 Étapes Guidées**

- **Étape 1 — Identité** : `full_name` , `date_of_birth` (DatePicker), `gender` (Dropdown), `national_id_number`
- **Étape 2 — Contact** : `phone_number` (lecture seule si OTP), `email` (lecture seule si Google)
- **Étape 3 — Localisation** : `city` , `region` , `address`
- Afficher une barre de progression avec % de complétion et un CTA clair pour chaque champ manquant.
- `date_of_birth` et `gender` doivent rester modifiables (erreur de saisie possible).

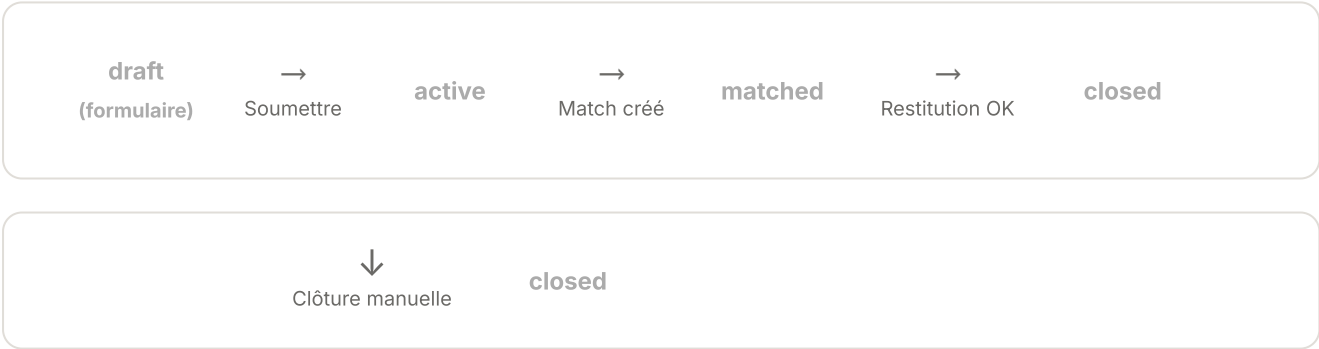
3.3 Module Déclarations (DECLARATIONS)

Une déclaration est soit un document **perdu** (`lost`) soit un document **trouvé** (`found`). C'est l'unité centrale autour de laquelle gravite tout le système.

Champs et Poids dans le Matching

Champ	Requis	Poids Matching	Notes
<code>declaration_type</code>	✓	—	"found" "lost"
<code>document_type</code>	✓	Gate (filtre préalable)	Enum : CNI, Passeport, Permis, etc.
<code>document_number</code>	—	+0.35 (hard gate si présent des 2 côtés)	Optionnel mais fortement conseillé
<code>owner_name</code>	—	+0.35 (Jaro-Winkler)	Tolérant aux fautes d'orthographe
<code>latitude / longitude</code>	—	+0.20 (Haversine 50km)	GPS auto via <code>location_service.dart</code>
<code>event_date</code>	—	+0.10 (cohérence <code>found_date ≥ lost_date</code>)	Date de perte ou de trouvaille
<code>photos</code>	—	—	Max 3 photos. Upload MinIO. Augmente la confiance.
<code>description</code>	—	—	Contexte libre, max 500 chars

Diagramme d'État — Déclaration



● Bug — Statut Non Mis à Jour lors du Match

- Dans `matching_service.py` , lorsqu'un `Match` est créé, les déclarations **ne passent PAS** à `status="matched"` .
- Elles restent `"active"` et peuvent générer de nouveaux matchs redondants.
- **Correction** : après `db.flush()` sur le match, setter `decl.status = "matched"` sur les deux déclarations impliquées.

⚠ Règle Anti-Fraude F-15 — Limite de 3 Déclarations Actives

- La limite de 3 déclarations actives simultanées est appliquée silencieusement (HTTP 429).
- L'utilisateur doit être prévenu AVANT de soumettre : afficher un compteur « **X / 3 déclarations actives** ».

- La limite doit être configurable par l'admin (paramètre de configuration, pas constante hardcodée).

3.4 Module OCR — Scan de Document

Le module OCR permet de photographier un document trouvé pour en extraire automatiquement les informations clés et pré-remplir le formulaire de déclaration.

Flux : Scan → Déclaration

1. L'utilisateur ouvre l'OCR depuis le bouton « Scanner » sur la page déclarations.
2. **OcrScanPage** : capture via caméra. Le bouton « Capturer » prend la photo.
3. Image envoyée au backend via `POST /ocr/extract` (backend_ocr_service.dart).
4. Backend analyse (regex + ML) et retourne : `document_type` , `document_number` , `owner_name` .
5. **OcrReviewPage** : affichage des résultats avec correction manuelle possible.
6. Validation → navigation vers **DeclarationFormPage** avec champs pré-remplis.

💡 Améliorations Recommandées

- Détection de qualité d'image avant envoi (flou, faible luminosité).
- Reconnaissance automatique du type de document (CNI vs Passeport vs Permis).
- Mode offline : extraction partielle par regex côté client si absence de réseau.

3.5 Module Matching — Algorithme de Correspondance

Le cœur de ReTurn. Déclenché automatiquement à chaque création de déclaration.

Score Multi-Critères (MAX = 1.0, SEUIL = 0.45)

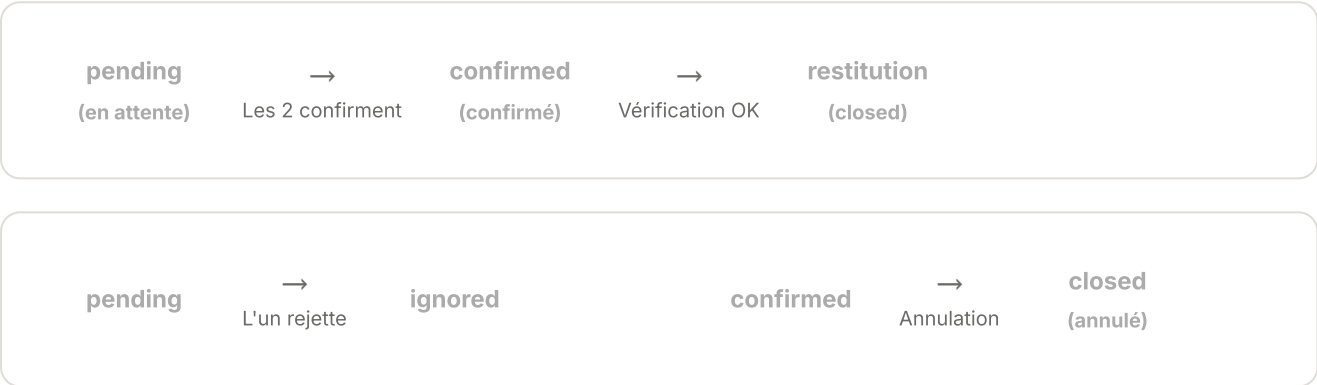
Numéro doc 0.35		Nom Jaro-Winkler 0.35		Géo Haversine 0.20	Temporal 0.10
Critère	Poids	Méthode	Détail		
Numéro de document	0.35	Égalité stricte (hard gate)	Si les deux numéros existent et diffèrent → score = 0 (disqualification immédiate). Un seul présent → 0.3 partiel.		
Nom du propriétaire	0.35	Jaro-Winkler similarity	Tolérant aux fautes d'orthographe, insensible à la casse. Noms normalisés en majuscules.		
Proximité géographique	0.20	Haversine (seuil 50 km)	Score décroît linéairement de 1.0 à 0.0 entre 0 et 50 km. 0 si coordonnées manquantes.		
Cohérence temporelle	0.10	found_date ≥ lost_date	1.0 si cohérent, 0.0 si incohérent, 0.5 si date manquante (neutre).		

⚠ Analyse Critique du Seuil MIN_SCORE = 0.45

- Sans numéro de document ET sans coordonnées GPS, le score max possible est : **nom (0.35) + temporal (0.10) = 0.45.**
- Un nom très similaire **suffit seul** pour créer un match — risque de faux positifs sur des homonymes.
- **Recommandation** : si les deux côtés sont sans numéro ET sans coordonnées, exiger `name_score ≥ 0.85` .
- Exposer un champ `score_confidence` dans l'API pour que l'admin puisse configurer le seuil dynamiquement.

3.6 Module Matches — Gestion des Correspondances

Diagramme d'État — Match



- **Problème — Double Confirmation Sémantiquement Ambiguë**
- Le modèle `Match` a `confirmed_by_owner` et `confirmed_by_finder` .
- Le modèle `Restitution` a **aussi** `confirmed_by_owner` et `confirmed_by_finder` .
- Ces champs encodent des concepts différents et créent de la confusion dans le code et les tests.
- **Solution** : Renommer → `Match` : `accepted_by_owner` / `accepted_by_finder` .
`Restitution` : `handoff_confirmed_by_owner` / `handoff_confirmed_by_finder`

3.7 Module Vérification d'Identité (VERIFICATION)

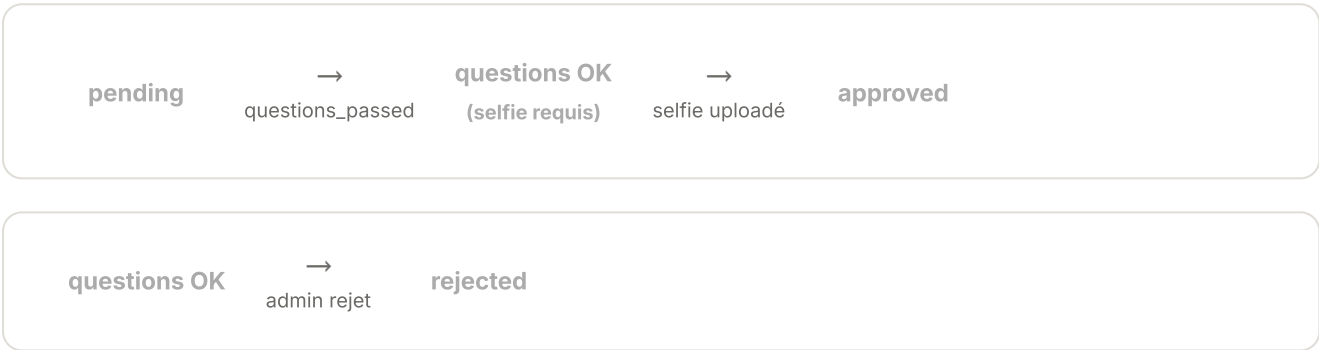
Avant toute restitution, le propriétaire supposé doit prouver qu'il est le titulaire légitime du document. C'est le garde-fou anti-fraude central.

Flux en 3 Étapes (UI Actuelle)

Étape	Label UI	Données Saisies	Traitement Backend
0 — Intro	Vérification rassurante	—	<code>GET /verify/{matchId}/status</code> → détermine l'étape de reprise

Étape	Label UI	Données Saisies	Traitement Backend
1 — Questions	Questions de contrôle	Nom complet + Numéro doc (facultatif !)	POST /verify/{matchId}/answers → compare avec la déclaration. questions_passed = true si OK.
2 — Selfie	Votre selfie	Photo frontale caméra avant	POST /verify/{matchId}/selfie → upload MinIO → status = "approved" automatiquement sans comparaison.
3 — Succès	Identité vérifiée ✓	—	Bouton → /restitution/{matchId}

Diagramme d'État — IdentityVerification



● Problèmes Critiques de Vérification — Sécurité Insuffisante

- **Auto-approbation aveugle** : le selfie est uploadé et `status = "approved"` *automatiquement*, sans comparaison biométrique ni revue humaine. N'importe qui connaissant le nom du propriétaire peut passer la vérification.
- **Numéro facultatif** : l'UI indique explicitement « laissez vide si vous ne l'avez pas ». La vérification peut donc passer avec *uniquement le nom*.
- **Qui vérifie ?** Le modèle `IdentityVerification` lie `(match_id, user_id)` mais rien n'empêche le mauvais utilisateur de lancer la vérification en premier.
- **Pas de revue admin** : aucun workflow pour qu'un administrateur examine des selfies suspects.

✓ Architecture de Vérification Recommandée — Niveau 3

- **Étape 1 — Questions obligatoires renforcées** : Nom exact + Date de naissance + Numéro si présent dans la déclaration (rendu obligatoire si disponible).
- **Étape 2 — Photo du document** : l'utilisateur photographie le document (recto). Hashage + stockage privé MinIO.
- **Étape 3 — Selfie avec liveness** : comparaison basique face detection côté backend.
- **Étape 4 — Approbation** : automatique si score confiance $\geq 85\%$, sinon revue admin obligatoire.
- Ajouter un champ `verification_level` : 1 (nom seul), 2 (nom+date), 3 (nom+date+numéro+selfie+photo doc).

3.8 Module Messagerie (MESSAGING)

Un chat temps réel est disponible entre les deux parties dès qu'un match est créé, permettant de coordonner la restitution sans exposer les données personnelles.

- Architecture WebSocket (`ws_manager.py` + Redis pour la gestion des connexions).
- Une room de chat par match : `room_id = match_id` .
- Historique des messages persisté en PostgreSQL (modèle `Message`).
- Notifications FCM envoyées si l'app est en arrière-plan.

💡 Améliorations Recommandées

- Indicateurs de lecture (lu / reçu) — absents actuellement.
- Rate limiting sur l'envoi de messages (max 10/min par utilisateur).
- Blocage automatique du chat si le match passe à `"ignored"` ou `"closed"` .
- Avertissement si l'utilisateur tente de partager un numéro de téléphone (anti-bypass).

3.9 Module Restitution (RESTITUTION)

Créée automatiquement quand les deux parties confirment le match et que la vérification est approuvée, la restitution orchestre la remise physique du document.

Diagramme d'État — Restitution



Données Clés d'une Restitution

- `meeting_location` + `meeting_lat/lon` : lieu de remise convenu.
- `proof_photos` : liste de photos de preuve (envoyées par finder et owner).
- `handoff_confirmed_by_owner` + `handoff_confirmed_by_finder` : double validation finale.
- `rating_by_owner` + `rating_by_finder` : note 1-5 étoiles post-remise → alimente `score_reputation` .
- `completed_at` : horodatage de la clôture.

📊 Calcul de la Réputation

- Après `completed` , les deux parties s'évaluent (1-5 étoiles).
- Formule :
$$\text{new_score} = (\text{old_score} \times n + \text{note} \times 2) / (n + 1)$$
 où n = nombre total de restitutions.
- La note est doublée pour la ramener sur l'échelle 0-10 du `score_reputation` .

3.10 Module Zones (ZONES)

Les zones de récupération sont des périmètres géographiques définis par l'admin (quartiers, communes, préfectures) qui permettent de contextualiser les déclarations et d'envoyer des alertes ciblées.

- Modèle `Zone` : `id`, `name`, `latitude`, `longitude`, `radius_km`, `admin_created` .
- Page `ZonesPage` : carte avec marqueurs. L'utilisateur peut s'abonner à une zone.
- Notification automatique si un document trouvé dans une zone d'abonnement.

⚠ Interface Admin des Zones Manquante

- Le modèle `Zone` existe en base (migration `004_admin_zones.py`) mais l'admin HTML n'a pas de section opérationnelle pour les gérer.
- À implémenter : CRUD zones dans l'admin + carte interactive + statistiques déclarations par zone.

3.11 Module Administration (ADMIN)

Le panneau admin est actuellement un unique fichier HTML (`admin/index.html`) avec Chart.js et du JavaScript vanilla. Interface visuelle avec sidebar, KPIs et graphiques — mais avec des limitations critiques.

● Faiblesses Critiques du Panneau Admin

- **Données mockées** : pas d'appels API réels visibles. Le tableau de bord ne reflète pas les vraies données en temps réel.
- **Authentification non dédiée** : l'admin utilise le même flux que les utilisateurs normaux (`is_admin` booléen). Pas de 2FA, pas de session admin séparée.
- **Rôles non hiérarchiques** : `is_admin` est un simple booléen. Pas de distinction super-admin / admin / modérateur.
- **Vérifications inaccessibles** : l'admin ne peut pas voir les selfies, approuver/rejeter les vérifications d'identité.
- **Pas de modération des signalements** : le modèle `Report` existe en base mais n'a pas d'interface de traitement.
- **Audit logs non affichés** : le modèle `AuditLog` existe mais ses données ne sont pas consultables.
- **Actions sensibles non confirmées** : ban, suppression, etc. sans confirmation multi-étapes.

Architecture Admin Cible

Section	Fonctionnalités Requises
Tableau de bord	KPIs temps réel (users J/S/M, matchs, restitutions, taux fraude). Graphiques d'évolution.
Gestion Utilisateurs	Liste paginée + recherche. Profil complet. Activer/désactiver/bannir. Changer <code>is_admin</code> . Voir réputation + historique.

Section	Fonctionnalités Requises
Gestion Déclarations	Filtres par statut/type/date. Voir les photos. Clôturer les déclarations abusives.
Gestion Matches	Voir les matches avec score. Forcer la fermeture d'un match frauduleux. Voir les vérifications associées.
Vérifications Identité	File de révision. Voir selfie + réponses. Approuver / Rejeter avec motif. Stats de fraude.
Modération Signalements	Liste des rapports. Décision (avertissement, suspension, suppression). Clôture des cas résolus.
Gestion Zones	Carte interactive. Créer/éditer/supprimer des zones. Stats par zone.
Audit Log	Timeline de toutes les actions admin. Filtrable. Export CSV.
Configuration	<code>ACTIVE_DECLARATIONS_LIMIT</code> , <code>MIN_SCORE</code> , délais d'expiration, paramètres FCM.

3.12 Module Notifications Push (FCM)

Événement	Destinataires	Contenu
Match créé	user_found + user_lost	« Un document correspondant a été trouvé ! Consultez le match. »
Match confirmé par l'autre partie	La partie non-confirmante	« L'autre partie a confirmé le match. À votre tour ! »
Vérification approuvée	user_lost (propriétaire)	« Identité vérifiée. Organisez la restitution. »
Nouveau message chat	Le destinataire	« [Nom] vous a envoyé un message. »
Restitution confirmée	Les deux parties	« Restitution confirmée ! Donnez une évaluation. »

4. FLUX UTILISATEUR GLOBAL

4.1 Parcours du Propriétaire Perdant (user_lost)

#	Écran / Action	Module	Condition de Suite
1	Splash → Vérification de session	AUTH	Si non authentifié → <code>/auth/phone</code>
2	Saisie numéro → OTP SMS	AUTH	Code OTP validé → session JWT créée
3	Onboarding (1ère fois)	SPLASH	Tour guidé des fonctionnalités
4	Compléter son profil	PROFILE	Nom + ville requis pour déclarer efficacement
5	Créer déclaration « lost »	DECLARATIONS	Type doc + nom + event_date minimum
6	Algorithme matching (automatique)	MATCHING	Score ≥ 0.45 → Match créé + notification push
7	Consulter le match	MATCHES	Voir score, infos document, retrouveur (anonymisé)
8	Confirmer le match	MATCHES	<code>accepted_by_owner = true</code>
9	Vérification d'identité	VERIFICATION	Questions + selfie → <code>approved</code>
10	Chat avec le retrouveur	MESSAGING	Coordonner lieu et heure de remise
11	Planifier la restitution	RESTITUTION	Saisir lieu + confirmer <code>handoff_confirmed_by_owner</code>
12	Évaluer le retrouveur	RESTITUTION	Note 1-5 → mise à jour <code>score_reputation</code>

4.2 Parcours du Retrouveur (user_found)

Symétrique au parcours du propriétaire, mais le retrouveur n'a pas à passer la vérification d'identité. Il doit :

1. Créer une déclaration « found » avec un maximum d'informations (photos incluses).
2. Confirmer le match (`accepted_by_finder = true`).
3. Participer au chat et confirmer le rendez-vous.
4. Uploader une photo de preuve lors de la remise physique.
5. Confirmer la restitution (`handoff_confirmed_by_finder`).

6. Évaluer le propriétaire (1-5 étoiles).

5. MODÈLE DE DONNÉES (ERD SIMPLIFIÉ)

Entité	Clé Primaire	Relations Clés	Statuts Possibles
users	UUID PK	→ declarations (1:N)	is_active, is_banned, is_admin, is_verified
declarations	UUID PK	→ user (N:1) · → matches (found ou lost)	active matched closed
matches	UUID PK	→ decl_found · → decl_lost · → user_found · → user_lost · → restitution (1:1)	pending confirmed ignored closed
identity_verifications	UUID PK	→ match (N:1) · → user (N:1)	pending approved rejected
restitutions	UUID PK	→ match (1:1 unique)	pending in_progress completed cancelled
messages	UUID PK	→ room (match_id) · → sender (user)	—
zones	UUID PK	→ admin (user créateur)	active inactive
reports	UUID PK	→ reporter (user) · → cible (user declaration)	open resolved dismissed
audit_logs	UUID PK	→ admin (user)	—
connection_logs	UUID PK	→ user	—

6. API BACKEND — ENDPOINTS CLÉS

Méthode	Endpoint	Auth	Description
POST	/api/v1/auth/firebase	Public	Échange idToken Firebase contre JWT backend
GET	/api/v1/auth/me	JWT	Profil utilisateur courant
POST	/api/v1/declarations/	JWT	Créer une déclaration (multipart + photos). Déclenche le matching.
GET	/api/v1/declarations/	JWT	Mes déclarations (cursor pagination)
DELETE	/api/v1/declarations/{id}	JWT	Supprimer une déclaration (mienne uniquement)
GET	/api/v1/matches/	JWT	Mes matchs (en tant que found ou lost)
POST	/api/v1/matches/{id}/confirm	JWT	Confirmer le match de mon côté
POST	/api/v1/matches/{id}/ignore	JWT	Ignorer / rejeter un match
GET	/api/v1/verify/{matchId}/status	JWT	Statut de ma vérification pour ce match
POST	/api/v1/verify/{matchId}/answers	JWT	Soumettre les réponses aux questions de contrôle
POST	/api/v1/verify/{matchId}/selfie	JWT	Uploader le selfie de vérification
GET	/api/v1/restitutions/{matchId}	JWT	Détail de la restitution liée à un match

Méthode	Endpoint	Auth	Description
POST	/api/v1/restitutions/{id}/confirm	JWT	Confirmer la remise physique
POST	/api/v1/restitutions/{id}/rate	JWT	Évaluer l'autre partie (1-5 étoiles)
WS	/ws/chat/{roomId}?token=...	JWT	WebSocket chat temps réel
GET	/api/v1/admin/users/	Admin JWT	Liste tous les utilisateurs
POST	/api/v1/admin/users/{id}/ban	Admin JWT	Bannir un utilisateur
POST	/api/v1/admin/verifications/{id}/approve	Admin JWT	Approuver manuellement une vérification d'identité
POST	/api/v1/ocr/extract	JWT	Extraire les données d'une image de document

7. SÉCURITÉ & CONFORMITÉ RGPD / CPDP

7.1 Mesures de Sécurité en Place

Mesure	Implémentation	Niveau
Authentification	Firebase Phone Auth + Google OAuth 2.0	✔ Fort
Tokens API	JWT HS256 (access + refresh), stockage SecureStorage	✔ Fort
Anti-fraude déclarations	Limite 3 déclarations actives simultanées (F-15)	✔ En place
Isolation des données	Chaque user ne voit que ses données (<code>user_id</code> filtre)	✔ OK
Stockage fichiers	MinIO privé (pas d'URLs publiques)	✔ OK
Biométrie locale	Verrouillage app par empreinte/Face ID (optionnel)	✔ Disponible
Refresh token auto	Non implémenté — déconnexion si access_token expire	⚠ Partiel
Vérification selfie	Upload sans comparaison biométrique	● Faible
Admin 2FA	Absent	● Absent
Rate limiting API	Partiel (seulement ACTIVE_DECLARATIONS_LIMIT)	⚠ Partiel
Chiffrement au repos	Tokens en SecureStorage, BD non chiffrée	⚠ Partiel

7.2 Droits RGPD / CPDP

- ✔ **Droit à l'effacement (F-04)** : Suppression complète du compte via l'API profil + suppression compte Firebase.
- ✔ **Portabilité des données** : données utilisateur accessibles via `GET /auth/me` .
- ⚠ **Droit de rectification** : Limité — `full_name` verrouillé une fois renseigné.
- ⚠ **Consentement explicite** : Absent d'un écran de consentement RGPD/CPDP à l'inscription.
- **Registre de traitements** : Non documenté dans l'application.
- **DPO désigné** : Non mentionné dans les CGU ou l'application.

8. ROADMAP & PRIORITÉS

8.1 Correctifs Critiques — Sprint 1 (2 semaines)

Priorité	Bug / Amélioration	Module	Effort
P0 ●	Mettre à jour <code>status = "matched"</code> sur les déclarations lors de la création d'un match	MATCHING	1 jour
P0 ●	Ajouter <code>date_of_birth</code> et <code>gender</code> dans le formulaire de profil	PROFILE	1 jour
P0 ●	Implémenter le refresh token automatique (intercepteur Dio 401 → POST /auth/refresh)	AUTH	2 jours
P0 ●	Renommer <code>confirmed_by_*</code> pour différencier Match (accepted) vs Restitution (handoff)	BACKEND	1 jour
P1▲	Afficher le compteur de déclarations actives (X/3) dans l'interface	DECLARATIONS	0.5 jour
P1▲	Exiger <code>date_of_birth</code> dans les questions de vérification	VERIFICATION	2 jours
P1▲	Externaliser <code>serverClientId</code> Google dans .env	AUTH	0.5 jour

8.2 Améliorations Majeures — Sprints 2 & 3 (4 semaines)

Priorité	Fonctionnalité	Module	Effort
P1	Refonte admin : API réelles, gestion vérifications, zones, rapports, audit log	ADMIN	2 semaines
P1	Vérification niveau 3 : photo du document + comparaison face detection backend	VERIFICATION	1 semaine
P2	Score matching adaptatif : <code>MIN_SCORE</code> configurable + seuil renforcé sans numéro de doc	MATCHING	3 jours
P2	Indicateurs de lecture dans le chat (lu/reçu)	MESSAGING	2 jours
P2	Barre de progression profil avec CTA par champ manquant	PROFILE	1 jour

Priorité	Fonctionnalité	Module	Effort
P2	Rate limiting global (slowapi ou nginx) + monitoring	INFRA	2 jours
P3	Écran de consentement RGPD/CPDP à l'inscription	AUTH	1 jour
P3	Interface Zones dans l'admin (carte + CRUD)	ZONES / ADMIN	3 jours
P3	Admin 2FA (TOTP) pour les comptes administrateurs	ADMIN / AUTH	3 jours

8.3 Évolutions Long Terme (3–6 mois)

- Partenariats avec les commissariats et bureaux d'état civil pour intégration officielle.
- Internationalisation complète : support de l'anglais (en cours), langues africaines (Bamileke, Ewondo...).
- Module Statistiques publiques : carte de chaleur des zones de perte.
- API ouverte pour les partenaires (assurances, administrations).
- Version web responsive pour accès sans smartphone.
- Programme de fidélité : badges et récompenses pour les retrouveurs réguliers.

9. GLOSSAIRE

Terme	Définition
Déclaration	Signalement d'un document perdu (<code>lost</code>) ou trouvé (<code>found</code>) par un utilisateur.
Match	Correspondance calculée entre une déclaration « lost » et une « found » dont le score $\geq$ MIN_SCORE.
Score de matching	Valeur entre 0.0 et 1.0 représentant la probabilité que deux déclarations concernent le même document.
Restitution	Processus de remise physique du document à son propriétaire, orchestré par l'application.
Vérification d'identité	Processus par lequel le propriétaire prouve qu'il est le titulaire du document via questions de contrôle et selfie.
user_lost	L'utilisateur ayant perdu le document (le propriétaire).
user_found	L'utilisateur ayant trouvé le document (le retrouveur).
Score Réputation	Note 0-10 calculée à partir des évaluations post-restitution. Défaut : 5.0.
Jaro-Winkler	Algorithme de similarité de chaînes, tolérant aux fautes de frappe. Utilisé pour comparer les noms.
Haversine	Formule de calcul de distance entre deux points GPS. Utilisée pour le score géographique.
FCM	Firebase Cloud Messaging — service de notifications push de Google.
MinIO	Serveur de stockage d'objets compatible Amazon S3, utilisé pour les photos et selfies.
CPDP	Commission de Protection des Données à caractère Personnel — équivalent camerounais de la CNIL française.
E.164	Format international de numéro de téléphone (+237XXXXXXXXXX pour le Cameroun).
Riverpod	Bibliothèque de gestion d'état Flutter, réactive et typée. Successeur de Provider.
Alembic	Outil de migration de base de données pour SQLAlchemy (Python).

Ce document est confidentiel et destiné exclusivement à l'équipe projet ReTurn / DocRetour.

Toute reproduction ou diffusion sans autorisation est strictement interdite.

© 2026 ReTurn — Tous droits réservés.